秘密ハイパーパラメータ推定を含む秘密ガウス過程回帰

橋本 順子[1,a)]　竹之内 大地[2]　五十嵐 大[1]　田中 哲士[1]　諸橋 玄武[1]

概要：本稿では機械学習の一手法であるガウス過程回帰（GPR）に着目し，学習・予測およびさらにそのハイパーパラメータの自動推定までを 3 パーティの秘密分散ベース秘密計算で高速に実現する方法を提案する．また実データを用いた実機評価を行い，平文と同等の予測精度が得られ，千件規模のデータでも全処理が 1 時間以内に完了することを報告する．

キーワード：秘密計算, 秘密分散, 秘密計算システム, ガウス過程回帰, ハイパーパラメータ推定

Secure Gaussian Process Regression with Secure Hyperparameter Estimation

Junko Hashimoto[1,a)]　Daichi Takenouchi[2]　Dai Ikarashi[1]　Satoshi Tanaka[1]
Gembu Morohashi[1]

Abstract: This paper focuses on Gaussian Process Regression (GPR), a machine learning method, and proposes an efficient method for performing its training, prediction, and even automatic hyperparameter estimation on three-party secret-sharing-based secure computation. We report an evaluation of our implementation on real-world datasets, confirming prediction accuracy equivalent to that of unencrypted computation and completing all processing within one hour for datasets of about one thousand records.

Keywords: Secure Computation, Secret Sharing, Secure Computation System, Gaussian Process Regression, Hyperparameter Estimation

1. はじめに

秘密計算（MPC: Multi-Party Computation）は暗号化[*1]したままデータを処理できる技術であり，近年は機械学習を要素技術とする AI への適用が研究されてきている [1], [2]．こうした機械学習手法の中でも，本稿ではガウス過程回帰に着目する．

機械学習による回帰分析では，一般に多くの学習データがあることが精度の前提となる．しかし，医療や材料開発のように，データを 1 件得るために臨床試験や実験を要する分野では，データ件数は数百〜数千件程度にとどまることが多い．ガウス過程回帰（GPR: Gaussian Process Regression）[3] は，このような少量のデータからでも精度よく回帰を行える手法であり，さらに，予測値そのもの（予測平均）だけでなく，その予測がどの程度確からしいかを表す予測分散を同時に出力する点に特徴がある．学習データが少ない入力領域では予測分散が大きくなるため，「次にどの条件で実験・測定を行えば最も多くの情報が得られるか」を予測分散に基づいて選ぶことができる．この考え方は実験計画法やベイズ最適化と呼ばれ，1 件あたりの試行コストが高い医療 [4]，創薬 [5]，材料科学 [6] といった分野で GPR が用いられる理由となっている．

これらの分野では，データ件数が少なく精度が得られないという課題がある．例えば希少疾患の臨床研究では，1 つの病院が保有する症例数が数十件程度の場合，十分な精度の予測モデルを構築できない．複数の病院のデータを統合

[1] NTT 社会情報研究所,
NTT Social Informatics Laboratories
[2] NTT アドバンステクノロジ株式会社,
NTT Advanced Technology Corporation
[a)] junko.hashimoto@ntt.com
[*1] ここでの暗号化とは秘密分散も含む広義の暗号化である

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

すれば件数が増え，精度の向上が期待できる．しかし，患者の臨床データは要配慮個人情報にあたり，セキュリティの観点からの法規制や倫理審査により病院間でのデータ統合は容易ではない．秘密計算はデータ保管だけでなく，処理時も含め常にデータが暗号化され誰もデータの平文に触れないシステムを実現するための要素技術であり，複数組織のデータ統合はその中でも最も期待される応用の一つである．さらにデータが少量で規模が比較的小さいことも，処理速度が課題である秘密計算と適合すると考えられる．

上述の考察を受けて本稿では，GPR の学習・予測およびさらにそのハイパーパラメータの自動推定までを秘密計算で実現する方法を提案する．特に，ハイパーパラメータの自動推定においては周辺尤度の評価を含み，(1) 出力スケールの解析的消去による探索次元の削減，(2) 階層グリッド探索による評価点数の削減，(3) コレスキー分解による逆行列計算コストの削減により高効率化を図る．

1.1 構成する秘密計算のモデル

本稿では，秘密計算の中でも最も速度に優れる，秘密分散ベースの 3 パーティ秘密計算を前提とする．また，秘密計算のモデルとして，複数の提供者自身が計算を担う参加型と，提供者が暗号化したデータを計算者に預け計算者が分析者の依頼に応じて計算を行い結果のみを返す委託型 [7] があるが，本稿では後者を想定する．暗号理論分野では参加型が想定されることが多いが，参加型では計算主体が組織をまたぐため実用時は WAN 通信となり，性能が低下する．委託型では，暗号化されたまま誰もデータの平文に触らないという特徴は変わらず，計算者内での内部不正や単一障害点を解消できるという十分なセキュリティ効果と LAN 等の高速通信を両立できる．

1.2 関連研究

データを秘密分散した計算サーバ上で GPR の学習・予測を行う研究 [8], [9] は存在するものの，ハイパーパラメータの推定を，3 パーティの秘密分散ベース秘密計算で実現した研究は確認できない．また，ハイパーパラメータの推定を周辺尤度の評価を含む全過程にわたって秘密計算上で実現した研究は確認できない．

準同型暗号を用いた研究としては，[10], [11], [12] が GPR を扱っているが，ハイパーパラメータの秘匿推定は行っていない．

秘密計算（MPC）に基づく研究としては，Luo et al.[8] およびその拡張である Liu et al.[9] が，2 台の計算サーバと 1 台の補助サーバからなる構成の秘密分散により，GPR の学習・予測を実現しているが，ハイパーパラメータの秘匿は行っていない．

なお，参加型モデルでは，Jang et al.[13] が，データ全体の周辺尤度を求めないハイパーパラメータ推定を行っている．これは，各エージェントが平文で自データセットから周辺尤度を求めてハイパーパラメータを推定し，秘密計算で平均値を求める方法であり，データセット間の交差共分散を考慮しない近似である．

1.3 本稿の貢献

本稿では，周辺尤度の評価を含む全過程を 3 パーティ秘密計算上で行う形で，GPR のハイパーパラメータ自動推定を実現した．GPR の入力データ（学習データ・テストデータ），ハイパーパラメータ，出力結果（予測平均・予測分散）をすべて秘密分散された値とし，計算の全過程を秘密計算サーバ上で暗号化したままで行った．また，3 つのデータセットによる評価を行い，予測精度・予測分散が平文（scikit-learn）と高精度で一致すること，および全処理が 1 時間以内に完了し，オフラインの学習ステップとして実用的であることを確認した．さらに，秘密分散を用いた GPR[8], [9] と比較し，ハイパーパラメータを秘匿したまま推定できることに加え，学習・予測についても同等以上の処理速度を達成することを示した．

2. 準備

2.1 ガウス過程回帰（GPR）

2.1.1 概要

GPR は，入力 $\vec{x}$ と出力 y の組からなる学習データをもとに，未知の入力に対する出力を予測する回帰手法である．線形回帰やニューラルネットワークが，学習によって関数の重みパラメータを定めるのに対し，GPR は関数の形をパラメータで固定せず，学習データそのものを保持したまま予測を行うノンパラメトリックな手法である．

GPR を構成する要素は次の 3 つである．

(i) カーネル関数 $k(\vec{x}, \vec{x}')$　2 つの入力がどれだけ「近い」かを測る関数であり，近い入力に対する出力は近い値をとる，という仮定を与える．

(ii) ハイパーパラメータ θ　カーネル関数の形状（どの程度離れたら「遠い」とみなすか，出力の振れ幅はどの程度か）と観測ノイズの大きさを規定する少数の値であり，学習データから推定する．

(iii) グラム行列 K　学習データ n 点の全点間のカーネル値を並べた $n \times n$ 行列であり，予測は K の逆行列を用いた計算として与えられる．

処理は，ハイパーパラメータ推定・学習（fit）・予測（predict）の 3 つからなる．ハイパーパラメータ推定は学習データ $(X, \vec{y})$ を入力として θ を出力する．学習は $(X, \vec{y}, \theta)$ を入力として予測に用いる係数を出力する．予測はテスト入力 $\vec{x}_*$ を入力として，予測平均 μ_* と予測分散 σ_*^2 を出力する．

GPR が他の多くの回帰手法と異なる点は，予測が単一の値ではなく分布（平均と分散）として得られることである．図 1 に模式図を示す．学習データが密に存在する入力

© 2026 Information Processing Society of Japan

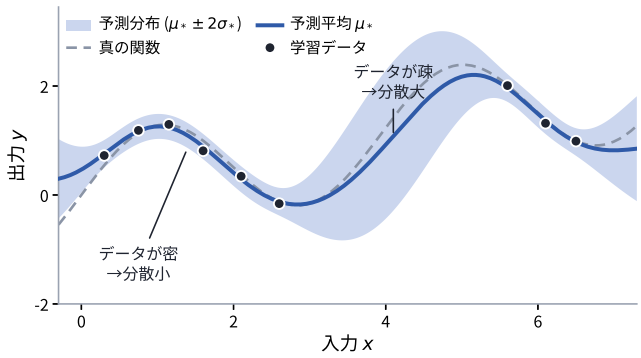


図 1 GPR の予測分布（模式図）．学習データが密な領域では予測分散が小さく，疎な領域では大きくなる．

領域では予測分散は小さく，学習データから離れた領域では予測分散は大きくなる．すなわち予測分散は，モデルがその予測をどれだけ確信しているかを表している．

2.1.2 事前分布・周辺尤度

d 次元の入力 $\vec{x} \in \mathbb{R}^d$ に対する未知関数 $f(\vec{x})$ を，ノイズ ε を伴って観測した n 件の学習データ $\mathcal{D} = \{(\vec{x}_i, y_i)\}_{i=1}^n$，$y_i = f(\vec{x}_i) + \varepsilon_i$，$\varepsilon_i \sim \mathcal{N}(0, \sigma_n^2)$（ホワイトノイズ）を考える．GPR は，$f$ に対して平均 0，共分散関数（カーネル）$k(\vec{x}, \vec{x}')$ を持つガウス過程の事前分布

$$f(\vec{x}) \sim \mathcal{GP}(0,\ k(\vec{x}, \vec{x}')) \tag{1}$$

を仮定する．学習データの入力を $X = (\vec{x}_1, \ldots, \vec{x}_n)^\top$，出力を $\vec{y} = (y_1, \ldots, y_n)^\top$ とし，$K := k(X, X)$ $(K_{ij} = k(\vec{x}_i, \vec{x}_j))$ をグラム行列とすると，データの周辺尤度は

$$\begin{aligned}\log p(\vec{y} \mid X, \theta) = &-\frac{1}{2}\vec{y}^\top (K + \sigma_n^2 I)^{-1}\vec{y} \\ &- \frac{1}{2}\log|K + \sigma_n^2 I| - \frac{n}{2}\log(2\pi)\end{aligned} \tag{2}$$

で与えられる．θ は，カーネル k および観測ノイズ分散 σ_n^2 を規定するハイパーパラメータである．ハイパーパラメータ推定では，式 (2) を最大化する θ を求める．

2.1.3 学習（fit）・予測（predict）

学習では，与えられたハイパーパラメータ θ に対し，学習データ $\mathcal{D}$ に対する双対係数 $\vec{\alpha}$ を求める．

$$\vec{\alpha} := (K + \sigma_n^2 I)^{-1}\vec{y} \tag{3}$$

予測では，未知のテスト入力 $\vec{x}_*$ に対する予測分布 $p(f(\vec{x}_*) \mid \mathcal{D}) = \mathcal{N}(\mu_*, \sigma_*^2)$ の平均 μ_*・分散 σ_*^2 を求める．

$$\mu_* = \vec{k}_*^\top \vec{\alpha}, \tag{4}$$

$$\sigma_*^2 = k(\vec{x}_*, \vec{x}_*) - \vec{k}_*^\top (K + \sigma_n^2 I)^{-1}\vec{k}_*, \tag{5}$$

ここで $\vec{k}_* := k(\vec{x}_*, X) = (k(\vec{x}_*, \vec{x}_i))_{i=1}^n$ は，テスト入力と学習データ各点との共分散ベクトルである．

2.1.4 RBF カーネル

本稿では，カーネルとして，長さスケール ℓ，出力スケール σ_f^2 を持つ動径基底関数（RBF）カーネル

$$k(\vec{x}, \vec{x}') := \sigma_f^2 \exp\left(-\frac{\|\vec{x} - \vec{x}'\|^2}{2\ell^2}\right) \tag{6}$$

を用いる．RBF カーネルは，入力空間上で滑らかな関数を表現でき，事前知識のない一般的な回帰問題における標準的な選択肢であることから採用した．このとき，ハイパーパラメータは $\theta = (\ell, \sigma_f^2, \sigma_n^2)$ の 3 つとなる．

2.1.5 出力スケールの解析的消去

式 (2) のハイパーパラメータ $\theta = (\ell, \sigma_f^2, \sigma_n^2)$ を推定するにあたり，σ_f^2 を解析的に消去することで探索対象を 2 変数に削減できる [14]．$\eta := \sigma_n^2/\sigma_f^2$（ノイズ対信号比）とおくと，$K + \sigma_n^2 I = \sigma_f^2(K_0 + \eta I) =: \sigma_f^2 K_\eta$（$K_0$ は $\sigma_f^2 = 1$ としたときのカーネル行列）と分解でき，式 (2) は

$$\begin{aligned}\log p(\vec{y} \mid X, \theta) = &-\frac{q}{2\sigma_f^2} - \frac{n}{2}\log\sigma_f^2 \\ &- \frac{1}{2}\log|K_\eta| + \text{const.}, \\ q := &\ \vec{y}^\top K_\eta^{-1}\vec{y}\end{aligned} \tag{7}$$

と書ける．σ_f^2 を解析的に最大化すると $\sigma_f^2 = q/n$ が得られ，これを式 (7) に代入することで，長さスケール ℓ を $\gamma := 1/(2\ell^2)$ と置いた上でのプロファイル尤度

$$f(\gamma, \eta) = -\frac{n}{2}\log q - \frac{1}{2}\log|K_\eta| \tag{8}$$

が得られる．探索対象は (γ, η) の 2 変数のみとなり，σ_f^2 は各評価点で q/n として自動的に定まる．

2.1.6 コレスキー分解

ハイパーパラメータ推定（式 (8)）には逆行列が含まれる．本稿では，$q = \vec{y}^\top K_\eta^{-1}\vec{y}$ だけではなく，式 (8) に必要な $\log|K_\eta|$ も同時に得られることから，コレスキー分解を採用する（他手法との比較は付録 A.1）．コレスキー分解は，対称正定値行列 K_η を下三角行列 L を用いて $K_\eta = LL^\top$ の形に分解する，数値線形代数における標準的な手法である．L は次式で求められる．

$$L_{jj} = \sqrt{(K_\eta)_{jj} - \sum_{k<j} L_{jk}^2} \tag{9}$$

$$L_{ij} = \frac{1}{L_{jj}}\left((K_\eta)_{ij} - \sum_{k<j} L_{ik}L_{jk}\right), \quad (i > j) \tag{10}$$

これにより，逆行列を直接計算せずに，連立一次方程式 $K_\eta\vec{a} = \vec{y}$ の解を前進代入・後退代入（付録 A.2）で求められる．$\vec{a} = K_\eta^{-1}\vec{y}$ より $q = \vec{y}^\top\vec{a}$ が得られる．また，行列式の対数も対角成分から直接得られる．

$$\log|K_\eta| = 2\sum_i \log L_{ii} \tag{11}$$

2.2 秘密計算

1.1 節の 3 パーティ秘密計算として，メルセンヌ素数 p を modulus とした素体上の $(2, 3)$-Shamir 秘密分散方式を

© 2026 Information Processing Society of Japan

用いる．攻撃者モデルは，プロトコルには従うが得られた情報から秘密を推測しようとする semi-honest（honest-but-curious）モデルを想定する．平文値 a を秘密分散した値を $[\![a]\!]$ と表記し，$[\![\vec{a}]\!]$，$[\![A]\!]$ は各要素が秘密分散されたベクトル・行列を表す．加減算およびスカラ倍はローカル計算のみで実行でき，乗算 $[\![a]\!] \times [\![b]\!]$[15] および右シフトは通信を伴う．実数は固定小数点表現により整数へ変換して秘密分散する（付録 A.3）．

本稿のアルゴリズムでは，以下の秘密計算関数を用いる．入出力はシェアであり，計算の過程で値を復元しない[*2]．

- $\mathsf{Exp}([\![x]\!])$，$\mathsf{Log}([\![x]\!])$，$\mathsf{SqrtInv}([\![x]\!])$：それぞれ e^x，$\log x$（$x>0$），$1/\sqrt{x}$（$x>0$）を計算する [16]．これらの初等関数は，効率的に計算するために入力・出力の値域を平文の属性としてあらかじめ与える必要がある．値域を明示する場合は，$\mathsf{Log}([\![x]\!];[l,u])$ のように第 2 引数（平文）で $l \le x \le u$ を与えるものと表記する．なお，底 10 のべき乗 $10^{[\![x]\!]}$ は，$\ln 10$ が公開値であることから $\mathsf{Exp}(\ln 10 \cdot [\![x]\!])$ として計算する．
- $\mathsf{Dot}([\![\vec{a}]\!],[\![\vec{b}]\!])$：内積 $\vec{a}^\top \vec{b}$ を計算する．引数が行列の場合は，対応する列同士の内積を列ごとに計算したベクトルを出力する．一方が $n \times m$ 行列 A，他方が長さ n のベクトル $\vec{b}$ の場合は，A の各列と $\vec{b}$ の内積を並べた長さ m のベクトル $A^\top \vec{b}$ を出力する．
- $[\![a]\!] > [\![b]\!]$：比較結果の真偽値 $[\![c]\!]$ を出力する．$\ge, <, \le$ も同様である．内部でビット分解 [17] を用いる．
- $\mathsf{IfElse}([\![flag]\!],[\![a]\!],[\![b]\!])$：$flag$ が真のとき $[\![a]\!]$，偽のとき $[\![b]\!]$ を出力する．$[\![flag]\!] \times ([\![a]\!] - [\![b]\!]) + [\![b]\!]$ により計算し，どちらが選択されたかは秘匿される．
- $\mathsf{RShift}([\![a]\!],t)$：$[\![a]\!]$ を t ビット右シフトした値（$\lfloor a/2^t \rfloor$）を出力する [18]．$[\![a]\!] \gg t$ とも表記する．

なお，行列・配列の並び替えを行う関数（At，Repeat，Tile）は値を計算しないためローカル計算のみで実行できる．定義は付録 A.3 に示す．

3. 提案手法

3.1 設計方針

提案手法は，学習データ・テストデータ・ハイパーパラメータ・出力（予測平均・予測分散）のすべてをシェアとして扱い，計算の全過程を通じて一切復元しない．ただし，学習データ件数 n，テストデータ件数 m，特徴量の次元数 d および固定値は，公開値として扱う．固定値は，階層グリッドの構造（段数・格子点数・初期探索中心・格子間隔）である．また，公開値のみから算出可能な値（値域など）も公開値とする．対応するカーネルは RBF カーネル（式 (6)）とする．GPR を秘密計算上で実現する際の技術課題に対し，以下の 3 つの方針で設計した．

[*2] ただし，値域および右シフトのシフト量は公開値である

(A) 計算の形をデータに依存させない．処理時間や通信パターンからデータの情報が漏れることを避けるため，反復回数がデータに依存する最適化手法（勾配法など）を用いず，固定回数の格子点評価で完結するグリッドサーチを採用する（3.3 節）．格子点間の比較・選択もシェアのまま行い，どの格子点が選ばれたかを秘匿する．

(B) 実数関数の値域を公開値のみで定める．初等関数に与える値域（2.2 節）にハイパーパラメータの値が反映されると，推定結果の秘匿性が損なわれる．本稿では，値域パラメータが常に公開値のみで定まるよう一貫して設計した．具体的には，ハイパーパラメータ推定（3.5 節）では値域をデータ件数の想定上限と探索範囲の下限のみから定め，学習（3.6 節）ではコレスキー分解の対象を K_η とし，予測（3.7 節）では学習時のコレスキー分解の結果を流用できるよう式変形を行った．

(C) 高コストな演算の呼び出し回数を削減する．GPR は $n \times n$ 行列の逆行列演算（$O(n^3)$）を要し，そこに現れる除算・平方根・初等関数は，秘密計算上では通信を伴う高コストな処理である．出力スケール σ_f^2 の解析的消去により探索を 2 変数に削減し（2.1 節），階層グリッド探索により評価点数を抑え（3.3 節），コレスキー分解では除算と平方根を $\mathsf{SqrtInv}$ の 1 回に統合する（3.4 節）．また，1 要素ずつ逐次処理するのではなく，可能な限り入出力をベクトルで与え一括処理する．

3.2 前処理とスケール

特徴量 X は各次元を平均 0・分散 1 に標準化した値を，目的変数 $\vec{y}$ は 0 ～ 1 の範囲に正規化した値を，シェアとして扱う．標準化・正規化はいずれもシェアのまま実行できる [7], [16]．また，計算の過程で小数点位置が変化した場合は，都度 24bit 固定小数点位置に戻して次の計算を行う．

3.3 ハイパーパラメータ推定：階層グリッド探索および出力スケールの解析的消去

本稿ではハイパーパラメータ推定にグリッドサーチを採用する．採用理由は次の 3 点である（他手法との比較は付録 A.4）．

1 点目は，3.1 節 (A) の通り，固定回数の格子点評価で完結し，計算の形がデータに依存しないことである．2 点目は，評価点数の増加を抑える見通しが立つことである．グリッドサーチは探索の次元数が多いほど，また探索範囲が広いほど評価点数が増える．前者に対しては 2.1 節の出力スケール σ_f^2 の解析的消去により探索対象を (γ,η) の 2 変数に削減し，後者に対しては，粗い格子で有望な範囲を絞り込んでから細かい格子で再探索する階層グリッド探索を用いる．3 点目は，探索範囲をデータに依存せず公開値のみから決定できることである．これにより，探索範囲の設定自体からの情報漏えいを避けつつ，実データの分布に即

© 2026 Information Processing Society of Japan

した効率的な初期範囲を得ることができる．

階層グリッド探索では，対数スケール上の格子探索を粗から細に 3 段階繰り返す．各段で 5×5 の格子上でスコア（式 (8)）を評価し，最良点を中心に次段の探索範囲を 1/4 に狭める．2 段目以降の探索範囲の中心は前段の最良点であり，シェアのまま保持されるため秘匿される．初期段の探索範囲の中心 c_γ, c_η と格子間隔 s_γ, s_η は平文で与える．γ については，3.2 節の通り入力 X を標準化しているため $E[\|\vec{x}_i - \vec{x}_j\|^2] \approx 2d$ が成り立ち，カーネルの指数部が $-2d\gamma = -1$ となる中心 $c_\gamma := -\log_{10}(2d)$ を次元数 d（公開値）のみから定め，$s_\gamma := 0.75$ とする．η については次元数に依存しないため，$c_\eta := -2$，$s_\eta := 1$（1 段目の探索範囲 $[10^{-4}, 1]$）とする．

3.4 ハイパーパラメータ推定：コレスキー分解

秘密計算において初等関数の計算は高コストなため，呼び出し回数を削減する．式 (9)，(10) は 2 種類の高コスト演算（平方根，除算）を含むが，$v := (K_\eta)_{jj} - \sum_{k<j} L_{jk}^2$ と置き，$L_{inv} := \mathsf{SqrtInv}(v)$ の 1 回の呼び出しに統合して

$$L_{jj} = v \times L_{inv} \tag{12}$$

$$L_{ij} = L_{inv}\left((K_\eta)_{ij} - \sum_{k<j} L_{ik}L_{jk}\right), \quad (i > j) \tag{13}$$

により求める．$L_{inv} = 1/\sqrt{v} = 1/L_{jj}$ であり，前進代入・後退代入における除算にもそのまま再利用できる．なお，SqrtInv の入力 v は K_η の対角成分（$1+\eta$）以下であるため，その値域は $A_S := [0, 1+\eta_{max}]$ と公開値のみで定まる．ここで $\eta_{max} = 10^{c_\eta + 2.625 s_\eta} = 10^{0.625}$ は，階層探索で到達しうる η の上限である．

3.5 ハイパーパラメータ推定：Log の値域

3.1 節 (B) の方針にもとづき，式 (8) の 2 か所の Log について，値域を公開値のみから定める．コレスキー因子の対角成分 L_{ii}（式 (9)）については，K_η の対角が $1+\eta$ であるため，$L_{ii} \le \sqrt{1+\eta} \le \sqrt{1+\eta_{max}}$ がグリッド全体で成り立つ（$A_L := [0, \sqrt{1+\eta_{max}}]$）．$q = \vec{y}^\top K_\eta^{-1} \vec{y}$ については，$\vec{y}$ が正規化されていること，K_0 が半正定値であることから $q \le n/\eta$ が成り立つ．したがって，データ件数の想定上限 n_{max}，探索範囲の下限 $\eta_{min} = 10^{-4.625}$ を用いて $q < n_{max}/\eta_{min}$ という，公開値のみで決まる上界が得られる（$A_q := [0, n_{max}/\eta_{min}]$）．

3.6 学習：コレスキー分解

学習においてもコレスキー分解を用いる．式 (3) の $K + \sigma_n^2 I$ をそのまま分解すると，その対角成分は $\sigma_f^2(1+\eta)$ となり，SqrtInv に与える値域が推定結果である σ_f^2 に依存してしまう．そこで本稿では，$\vec{\alpha}$ の代わりに $\vec{\beta} := \sigma_f^2 \vec{\alpha}$ を求める．

$$\vec{\beta} := K_\eta^{-1}\vec{y}, \qquad K_\eta = K_0 + \eta I \tag{14}$$

K_η の対角は常に $1+\eta$ であり σ_f^2 に依存しないため，学習においても SqrtInv の値域 A_S を用いることができる．

3.7 予測：コレスキー分解

予測では，式 (4)，(5) により μ_*，σ_*^2 を求める．この式を，$K^{-1} = (1/\sigma_f^2)K_\eta^{-1}$ および $\vec{k}_* = \sigma_f^2 \vec{k}_{0,*}$（$\vec{k}_{0,*}$ は $\sigma_f^2 = 1$ で構築した交差共分散）を用いて変形し，学習と同様に K_η をコレスキー分解の対象とする．

$$\mu_* = \vec{k}_{0,*}^\top \vec{\beta} \tag{15}$$

$$\sigma_*^2 = \sigma_f^2\left(1 - \vec{k}_{0,*}^\top K_\eta^{-1} \vec{k}_{0,*}\right) \tag{16}$$

学習で計算したコレスキー因子 L を予測の入力とすることで，予測でのコレスキー分解を不要とすることができる．

3.8 提案アルゴリズム

ハイパーパラメータ推定・学習・予測の各アルゴリズムを示す．X は学習データであり，n 点の d 次元特徴量からなる $n \times d$ の行列である．$\vec{y}$ は学習データの出力（目的変数）である．X_* はテストデータであり，m 点の d 次元特徴量からなる $m \times d$ の行列である．ハイパーパラメータは，$\gamma := 1/(2\ell^2)$，$\eta := \sigma_n^2/\sigma_f^2$，$\sigma_{f2} := \sigma_f^2$ と表記する．K_η はノイズを加えたカーネル行列，$K_{0,*}$ は学習データとテストデータの間の交差共分散行列で，出力スケール $\sigma_f^2 = 1$ としたものである．予測平均は $\vec{\mu}_*$，予測分散は $\vec{\sigma}_{*2} := \vec{\sigma}_*^2$ と表記する．カーネル行列の構築（BuildK_0，$\mathsf{BuildCrossK}_0$），コレスキー分解（Cholesky）および前進後退代入（Solve）は，付録 A.5 に示す．

3.8.1 ハイパーパラメータ推定アルゴリズム

階層グリッド探索によるハイパーパラメータ推定のアルゴリズムを Algorithm 1 に示す．初期探索範囲の中心 c_γ, c_η および格子間隔 s_γ, s_η は，3.3 節の通り，c_γ は次元数 d から，他は定数として平文で算出し入力する．これらは step 2 でシェアに変換し，その後はシェアのまま扱う．そのため，各格子点のスコアや，どの格子点が選ばれたかなど，探索の途中経過のすべては秘匿される．値域 A_q，A_L，A_S は，3.4 節および 3.5 節の通り，固定値および公開値 n_{max}，η_{min}，η_{max} から定まる値を入力する．最終的な出力 $[\![\gamma]\!]$，$[\![\eta]\!]$ は 3 段目で式 (8) が最大となった γ, η であり，$[\![\sigma_{f2}]\!]$ はそのときの q/n である．

3.8.2 学習・予測アルゴリズム

学習のアルゴリズムを Algorithm 2 に，予測のアルゴリズムを Algorithm 3 に示す．予測では m 個のテスト点を一括で処理する．$[\![\vec{\beta}]\!], [\![L]\!], [\![\vec{L}_{inv}]\!]$ は学習の出力をそのまま再利用しており，予測ではコレスキー分解を行わない．

© 2026 Information Processing Society of Japan

Algorithm 1 HyperparamEstimation

Functionality: $[\![\gamma]\!], [\![\eta]\!], [\![\sigma_{f2}]\!]$
$\leftarrow \mathsf{HyperparamEstimation}([\![X]\!], [\![\vec{y}]\!], n, d, c_\gamma, c_\eta, s_\gamma, s_\eta, A_q, A_L, A_S)$
Input: $[\![X]\!]$, $[\![\vec{y}]\!]$, n, d, $c_\gamma, c_\eta, s_\gamma, s_\eta$, A_q, A_L, A_S（値域，平文）
Output: $[\![\gamma]\!]$, $[\![\eta]\!]$, $[\![\sigma_{f2}]\!]$，ただし $\sigma_{f2} = q/n$

$[\![\widehat{score}]\!] := -\infty$ // 十分小さい値
$([\![c_\gamma]\!], [\![c_\eta]\!], [\![s_\gamma]\!], [\![s_\eta]\!]) := (c_\gamma, c_\eta, s_\gamma, s_\eta)$
for $lv = 0, 1, 2$ do // 3 段階の階層グリッド
  for all $(a, b) \in \{0, \ldots, 4\}^2$ do // 5×5 グリッド
    $off_a := a - 2$； $off_b := b - 2$
    $[\![c'_\gamma]\!] := [\![c_\gamma]\!] + off_a \cdot [\![s_\gamma]\!]$； $[\![c'_\eta]\!] := [\![c_\eta]\!] + off_b \cdot [\![s_\eta]\!]$
    $[\![\gamma]\!] := 10^{[\![c'_\gamma]\!]}$； $[\![\eta]\!] := 10^{[\![c'_\eta]\!]}$
    $[\![K_\eta]\!] := \mathsf{BuildK}_0([\![X]\!], n, d, [\![\gamma]\!]) + [\![\eta]\!] \cdot I$
    $[\![L]\!], [\![\vec{L}_{inv}]\!] := \mathsf{Cholesky}([\![K_\eta]\!], A_S)$
    $[\![\vec{a}]\!] := \mathsf{Solve}([\![L]\!], [\![\vec{L}_{inv}]\!], [\![\vec{y}]\!])$
    $[\![q']\!] := \mathsf{Dot}([\![\vec{y}]\!], [\![\vec{a}]\!])$
    $[\![score']\!] := -(n \cdot \mathsf{Log}([\![q']\!]; A_q) \gg 1) - \sum_i \mathsf{Log}([\![L_{ii}]\!]; A_L)$ // 式 (8)，(11)
    $[\![flag]\!] := ([\![score']\!] > [\![\widehat{score}]\!])$
    $([\![\widehat{score}]\!], [\![\hat{c}_\gamma]\!], [\![\hat{c}_\eta]\!], [\![\hat{q}]\!]) := \mathsf{IfElse}([\![flag]\!], ([\![score']\!], [\![c'_\gamma]\!], [\![c'_\eta]\!], [\![q']\!]), ([\![\widehat{score}]\!], [\![\hat{c}_\gamma]\!], [\![\hat{c}_\eta]\!], [\![\hat{q}]\!]))$
  $([\![c_\gamma]\!], [\![c_\eta]\!]) := ([\![\hat{c}_\gamma]\!], [\![\hat{c}_\eta]\!])$
  $[\![s_\gamma]\!] := [\![s_\gamma]\!] \gg 2$； $[\![s_\eta]\!] := [\![s_\eta]\!] \gg 2$
$[\![\gamma]\!] := 10^{[\![\hat{c}_\gamma]\!]}$； $[\![\eta]\!] := 10^{[\![\hat{c}_\eta]\!]}$
$n_{inv} := 1/n$ // 平文で算出
$[\![\sigma_{f2}]\!] := [\![\hat{q}]\!] \times n_{inv}$
Output $[\![\gamma]\!]$, $[\![\eta]\!]$, $[\![\sigma_{f2}]\!]$

Algorithm 2 Fit

Functionality: $[\![\vec{\beta}]\!], [\![L]\!], [\![\vec{L}_{inv}]\!]$
$\leftarrow \mathsf{Fit}([\![X]\!], [\![\vec{y}]\!], n, d, [\![\gamma]\!], [\![\eta]\!], A_S)$
Input: $[\![X]\!]$, $[\![\vec{y}]\!]$, n, d, $[\![\gamma]\!]$, $[\![\eta]\!]$, A_S
Output: $[\![\vec{\beta}]\!]$，ただし $\vec{\beta} = K_\eta^{-1}\vec{y}$
$[\![L]\!]$，ただし $K_\eta = LL^\top$
$[\![\vec{L}_{inv}]\!]$，ただし $L_{inv,j} := 1/L_{jj}$

$[\![K_\eta]\!] := \mathsf{BuildK}_0([\![X]\!], n, d, [\![\gamma]\!]) + [\![\eta]\!] \cdot I$
$[\![L]\!], [\![\vec{L}_{inv}]\!] := \mathsf{Cholesky}([\![K_\eta]\!], A_S)$
$[\![\vec{\beta}]\!] := \mathsf{Solve}([\![L]\!], [\![\vec{L}_{inv}]\!], [\![\vec{y}]\!])$
Output $[\![\vec{\beta}]\!]$, $[\![L]\!]$, $[\![\vec{L}_{inv}]\!]$

Algorithm 3 Predict

Functionality: $[\![\vec{\mu}_*]\!], [\![\vec{\sigma}_{*2}]\!]$
$\leftarrow \mathsf{Predict}([\![X]\!], [\![\vec{\beta}]\!], [\![L]\!], [\![\vec{L}_{inv}]\!], n, d, [\![X_*]\!], m, [\![\gamma]\!], [\![\sigma_{f2}]\!])$
Input: $[\![X]\!]$, $[\![\vec{\beta}]\!]$, $[\![L]\!]$, $[\![\vec{L}_{inv}]\!]$, n, d, $[\![X_*]\!]$, m, $[\![\gamma]\!]$, $[\![\sigma_{f2}]\!]$
Output: $[\![\vec{\mu}_*]\!]$，ただし $\mu_* = \vec{k}_{0,*}^\top \vec{\beta}$
$[\![\vec{\sigma}_{*2}]\!]$，ただし $\sigma_{*2} = \sigma_{f2}(1 - \vec{k}_{0,*}^\top K_\eta^{-1} \vec{k}_{0,*})$

$[\![K_{0,*}]\!] := \mathsf{BuildCrossK}_0([\![X]\!], n, [\![X_*]\!], m, d, [\![\gamma]\!])$
$[\![\vec{\mu}_*]\!] := \mathsf{Dot}([\![K_{0,*}]\!], [\![\vec{\beta}]\!])$ // 式 (15)
$[\![W]\!] := \mathsf{Solve}([\![L]\!], [\![\vec{L}_{inv}]\!], [\![K_{0,*}]\!])$ // m 列同時
$[\![\vec{kw}]\!] := \mathsf{Dot}([\![K_{0,*}]\!], [\![W]\!])$
$[\![\vec{\sigma}_{*2}]\!] := [\![\sigma_{f2}]\!] \times (\vec{1} - [\![\vec{kw}]\!])$
$[\![\vec{\sigma}_{*2}]\!] := \mathsf{IfElse}([\![\vec{\sigma}_{*2}]\!] < [\![\vec{0}]\!], [\![\vec{0}]\!], [\![\vec{\sigma}_{*2}]\!])$ // 0 以上
Output $[\![\vec{\mu}_*]\!]$, $[\![\vec{\sigma}_{*2}]\!]$

4. 評価

4.1 システム構成・環境

秘密計算システムとして，MEVAL3[7] を用いた．MEVAL3 は，サーバクライアント型のシステムであり，秘密計算クライアント上に秘密計算プログラムを定義し，3 台の秘密計算サーバへリクエストを送信して，秘密計算サーバが秘密計算を実行し，秘密計算クライアントへ計算結果のシェアを返却する．

オンプレミス環境上に，秘密計算サーバ 3 台（リング接続）および秘密計算クライアント 1 台を構築した．各機器は，Intel Xeon Gold 6334 3.60GHz（8 コア/16 スレッド）×2，メモリ 1TB，Rocky Linux 9.2 の構成である．秘密計算サーバ間の通信には 10Gbps NIC を，秘密計算クライアント・サーバ間の通信には 1Gbps NIC を用いた．測定は 3 回行い，平均値を記載する．

4.2 利用データ

評価には，diabetes[19]，Concrete Compressive Strength（CCS）[20]，Airfoil Self-Noise[21] の 3 つのデータセットを用いた．各データセットをシャッフルした上で 8:2 の比率で学習データ・テストデータに分割し，平文（scikit-learn）と秘密計算（本実装）で同一のデータを用いて評価した．各データセットの属性数・件数を表 1 に示す．各データセットは 3.2 節に記載した前処理を行ったシェアを入力としており，標準化・正規化の処理時間は本稿の測定結果に含まない．

表 1 データセット概要

	diabetes	ccs	airfoil
属性数 d	10	8	5
学習データ件数	354	824	1202
テストデータ件数	88	206	301

4.3 精度

平文（scikit-learn）と秘密計算（本実装）でハイパーパラメータ推定・学習・予測を行い，結果を比較した．ハイパーパラメータ推定の結果を表 2 に示す．長さスケール ℓ，ノイズ対信号比 η および観測ノイズ分散 σ_n^2 は，平文・秘密計算で近い値に収束している．σ_f^2 のみ最大 7%程度の差があるが，これは階層グリッドの格子間隔に由来する量子化誤差である．

予測精度については，R^2 スコアの差は最大 0.0009 であることが確認できた（表 3）．また，テスト点ごとに平文と秘密計算の予測値を比較すると，予測平均 μ_* の誤差は μ_* の標準偏差の 1%未満（0.41〜0.57%），予測標準偏差の

© 2026 Information Processing Society of Japan

誤差は RMSE で 0.005〜0.035（μ_* と同一スケール）に収まっており（付録 A.6），σ_f^2 の差が予測結果に与える影響は軽微であることを確認した．

表 2 ハイパーパラメータ推定の比較（ℓ，η は「平文 / 秘密計算」，σ_f^2，σ_n^2 は秘密計算の平文に対する相対差）

	diabetes	ccs	airfoil
ℓ	10.86 / 10.94	2.778 / 2.828	1.893 / 1.902
η	0.0928 / 0.0866	0.00917 / 0.00866	0.00102 / 0.00100
σ_f^2（差）	+6.96%	+7.23%	+1.98%
σ_n^2（差）	−0.17%	+1.30%	+0.09%

表 3 予測精度（R^2）の比較

	score_train		score_test	
	平文	秘密計算	平文	秘密計算
diabetes	0.5119	0.5126	0.6163	0.6163
ccs	0.9422	0.9413	0.9157	0.9155
airfoil	0.9112	0.9112	0.8698	0.8694

4.4 処理速度

各処理の所要時間を表 4 に示す．コレスキー分解は理論上 $O(n^3)$ であるが，本稿で扱う件数域では通信オーバーヘッドの影響が相対的に大きいため，実測の処理時間は理論値より緩やかに増加する．実データにおける最大の処理時間（Airfoil，$n = 1202$）でも，ハイパーパラメータ推定は約 34 分，fit は約 35 秒，predict は約 70 秒であり，全処理で約 36 分である．1 節で述べた GPR が実際に活用される分野では 1 件のデータ取得に実験や臨床試験を要するため，この処理時間は次の試行までのサイクルに対して十分短い．3.1 節 (C) の効果として，出力スケールの解析的消去により格子点数は 5^3 点から 5^2 点/段へ，階層グリッド探索により単一格子の 65^2 点から計 75 点へ削減され，コレスキー分解では高コストな初等関数の呼び出しが $O(n^2)$ から $O(n)$ となる．また，理論計算量との整合性をダミーデータで検証したところ，n の増加とともに実効指数は上昇し，$n = 2000$ では理論値 $O(n^3)$ に向かって漸近する挙動を示す一方，d への依存はほとんど見られなかった（付録 A.6）．

表 4 処理時間（秒）

処理	diabetes	ccs	airfoil
ハイパーパラメータ推定	330.62	1064.18	2035.03
fit	6.54	19.33	34.61
predict（テストデータ）	4.26	26.16	69.19

表 5 既存研究との構成の比較

項目	[8],[9]	本手法
秘密分散	$(2,2)$-加法的秘密分散．計算サーバ 2 台＋補助サーバ 1 台	$(2,3)$-Shamir 秘密分散．サーバ 3 台，補助サーバ不要
実行環境	[8]:Xeon 48 コア 2.9GHz，LAN 帯域 625MB/s ／ [9]:Tesla V100 GPU×3，LAN 帯域 10GB/s	Xeon Gold 6334 3.60GHz（GPU 不使用），10Gbps NIC
カーネル	RBF（[8] は Matérn 3/2 にも対応，[9] は次元ごとに独立な長さスケール）	RBF（長さスケール共通）
ハイパーパラメータ	平文で事前決定し，計算サーバ間で**公開値**として共有	秘密計算で推定し，**シェアのまま**保持
逆行列	コレスキー分解により明示的に構成	明示的には構成せず，前進後退代入で代替
指数関数	マスクした値を復元．情報漏えい量を定量化した上で許容	復元しない [16]

4.5 既存研究との比較

秘密分散を用いた GPR を扱った既存研究である Luo et al.[8] および Liu et al.[9]（[8] の拡張版）と本手法を比較する．両者の構成の比較を表 5 に示す．

[8][9] はハイパーパラメータ $(\ell, \sigma_f^2, \sigma_n^2)$ を計算サーバ間の公開値として扱っている．ハイパーパラメータは学習データの統計的性質を反映した値であり，これを公開することはデータに関する情報の漏えいにつながる．本手法は，探索過程を含めて一切復元せず，シェアのまま学習・予測に引き渡す．

処理時間の比較を表 6 に示す．[9] の値は事前計算・モデル構築・推論の全段階を含む総時間であるため，本手法の fit と predict の合計と対比した．なお，[9] の学習・テストの分割比率は不明なため合わせられなかった．

表 6 同一データセットでの処理時間の比較（秒）

	[8]	[9]	本手法
fit + predict			
diabetes	97.1*1	30.6	**10.80**
Airfoil	—	126*2	**103.80**
ハイパーパラメータ推定			
diabetes	非対応	非対応	330.62
Airfoil	非対応	非対応	2035.03

各手法の実行環境（表 5）が異なるため，厳密な比較ではない．*1: [8] は $n = 300$，$m = 142$ での測定値（本稿は $n = 354$，$m = 88$）．モデル構築と予測の両段階を含む実時間（計算時間・通信時間を含む）である．*2: [9] の低ランク近似版 Split-GP では 15.3 秒だが，RMSE が 0.137 から 0.333 へ劣化する．本比較はカーネル行列を近似しない厳密な GPR 同士の比較である．

© 2026 Information Processing Society of Japan

[8] との比較では，学習データ件数が多い（354 件対 300 件）にもかかわらず，本手法が数倍高速である．また [9] は全計算サーバに Tesla V100 GPU を搭載し，本手法の約 8 倍の帯域（10GB/s 級）のネットワークを用いているが，GPU を使用しない本手法はこれと同等以上の処理速度を実現している．

5. おわりに

本稿では，3 パーティの秘密分散ベース秘密計算で GPR のハイパーパラメータ自動推定・学習・予測を高速に実現する方法を提案した．出力スケール σ_f^2 の解析的消去，階層グリッド探索，コレスキー分解による統一的な定式化により，秘密計算特有の制約の下でも安定に動作する方式を実現した．

3 データセットによる評価を行い，予測精度・予測分散が平文（scikit-learn）と高精度で一致すること，および GPR が実際に活用される数百〜千件規模のデータに対して，ハイパーパラメータ自動推定を含めて全処理が 1 時間以内に完了し，オフラインの学習ステップとして実用的であることを確認した．

また，秘密分散を用いた GPR[8], [9] との比較を行い，カーネル行列を近似しない厳密な GPR という同一条件の下で同等以上の処理速度を達成すること，および既存研究が公開値として扱っていたハイパーパラメータを秘匿したまま推定できることを示した．

今後の課題は，RBF カーネル・ホワイトノイズ以外への対応，更なる高速化，より大規模なデータセットへの対応である．

参考文献

[1] N. Attrapadung, K. Hamada, D. Ikarashi, R. Kikuchi, T. Matsuda, I. Mishina, H. Morita, J. C. N. Schuldt. Adam in Private: Secure and Fast Training of Deep Neural Networks with Adaptive Moment Estimation. *Proceedings on Privacy Enhancing Technologies*, 2022(4):746-767, 2022.

[2] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. E. Lauter, M. Naehrig, J. Wernsing. CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, PMLR 48:201-210, 2016.

[3] C. E. Rasmussen, C. K. I. Williams. Gaussian Processes for Machine Learning. MIT Press, 2006.

[4] J. Willard, S. Golchi, E. E. M. Moodie, B. Boulanger, B. P. Carlin. Bayesian Optimization for Personalized Dose-Finding Trials with Combination Therapies. *Journal of the Royal Statistical Society Series C: Applied Statistics*, 74(2):373-390, 2025.

[5] R.-R. Griffiths, L. Klarner, H. Moss, et al. GAUCHE: A Library for Gaussian Processes in Chemistry. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36, pp. 76923–76946, 2023.

[6] V. L. Deringer, A. P. Bartók, N. Bernstein, D. M. Wilkins, M. Ceriotti, G. Csányi. Gaussian Process Regression for Materials and Molecules. *Chemical Reviews*, 121(16):10073-10141, 2021.

[7] 桐淵 直人，五十嵐 大，濱田 浩気，菊池 亮．プログラマブルな秘密計算ライブラリ MEVAL3. In SCIS, 2018.

[8] J. Luo, Y. Zhang, J. Zhang, S. Qin, H. Wang, Y. Yu, Z. Xu. Practical Privacy-Preserving Gaussian Process Regression via Secret Sharing. In *Proceedings of the Thirty-Ninth Conference on Uncertainty in Artificial Intelligence (UAI)*, PMLR 216:1315-1325, 2023.

[9] S. Liu, J. Luo, Y. Zhang, H. Wang, Y. Yu, Z. Xu. Efficient Privacy-Preserving Gaussian Process via Secure Multi-Party Computation. *Journal of Systems Architecture*, 151:103134, 2024.

[10] P. Fenner, E. O. Pyzer-Knapp. Privacy-Preserving Gaussian Process Regression – A Modular Approach to the Application of Homomorphic Encryption. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(04):3866-3873, 2020.

[11] 前嶋 啓彰，小林 鉄太郎，星野 文学．準同型暗号におけるガウス過程回帰の補助変数法による高速化．In SCIS, 2026.

[12] A. Nawaz, G. Chen, M. U. Raza, Z. Iqbal, J. Li, V. C. M. Leung, J. Chen. Secure Distributed Sparse Gaussian Process Models Using Multi-Key Homomorphic Encryption. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(13):14431-14439, 2024.

[13] Y. Jang, K. Teranishi, J. Suh, T. Tanaka. Privacy-Preserving Fully Distributed Gaussian Process Regression. arXiv:2512.05473, 2025.

[14] J. Sacks, W. J. Welch, T. J. Mitchell, H. P. Wynn. Design and Analysis of Computer Experiments. *Statistical Science*, 4(4):409-423, 1989.

[15] R. Gennaro, M. O. Rabin, and T. Rabin. Simplified VSS and Fast-Track Multiparty Computations with Applications to Threshold Cryptography. In PODC, pp. 101–111, 1998.

[16] 五十嵐 大. M op/s を超える秘密計算上の初等関数群. In SCIS, 2020.

[17] R. Kikuchi, D. Ikarashi, T. Matsuda, K. Hamada, K. Chida. Efficient Bit-Decomposition and Modulus-Conversion Protocols with an Honest Majority. In *ACISP*, LNCS 10946, pp. 64–82, 2018.

[18] 五十嵐 大. 秘密計算 AI の実装に向けた秘密実数演算群の設計と実装 — $O(|p|)$ ビット通信量 $O(1)$ ラウンドの実数向け右シフト —. In CSS, pp. 1557–1564, 2019.

[19] B. Efron, T. Hastie, I. Johnstone, R. Tibshirani. Least Angle Regression. *The Annals of Statistics*, 32(2):407-499, 2004.

[20] I-C. Yeh. Modeling of Strength of High-Performance Concrete Using Artificial Neural Networks. *Cement and Concrete Research*, 28(12):1797-1808, 1998. https://doi.org/10.24432/C5PK67

[21] T. F. Brooks, D. S. Pope, M. A. Marcolini. Airfoil Self-Noise and Prediction. NASA Reference Publication 1218, 1989. https://doi.org/10.24432/C5VW2C

付　　録

A.1 逆行列計算手法

ハイパーパラメータ推定（式 (8)）には逆行列が含まれる．逆行列を求める一般的な手法を表 A·1 に示す．本稿で

© 2026 Information Processing Society of Japan

は，$q = \vec{y}^\top K_\eta^{-1}\vec{y}$ だけではなく，式 (8) に必要な $\log|K_\eta|$ も得られることから，コレスキー分解を採用する．

表 A·1 行列計算手法と秘密計算での課題/メリット

手法	概要	秘密計算での課題/メリット
コレスキー分解	対称正定値行列の直接分解，$K_\eta = LL^\top$	通信ラウンド数・除算/平方根回数が予測しやすく，$\log\|K_\eta\|$ も同時に得られる（**採用**）
CG 法	行列ベクトル積の反復により $K_\eta^{-1}\vec{y}$ を近似	反復回数がデータに依存し不定．$\log\|K_\eta\|$ が別途必要
LU 分解	対称性を用いない一般的な直接法	コレスキー分解より計算量・メモリが約 2 倍．対称性を活かせない
Sparse GP（誘導点法）	$m \ll n$ 個の誘導点により $O(n^3)$ を $O(nm^2)$ に削減	誘導点の選択がデータに依存し，近似誤差が予測分散に影響する
確率的トレース推定+Lanczos	ランダムベクトルによる $\log\|K\|$ の確率的近似	秘匿乱数の生成が必要で，反復回数がデータに依存

A.2 前進代入・後退代入の式

本文 2.1.6 節で述べた通り，逆行列を直接計算せずに，連立一次方程式 $K_\eta\vec{a} = \vec{y}$ の解を前進代入・後退代入で求められる．前進代入では $L\vec{z} = \vec{y}$ を，後退代入では $L^\top\vec{a} = \vec{z}$ を，それぞれ次式により解く．

$$z_i = \frac{1}{L_{ii}}\left(y_i - \sum_{k<i} L_{ik}z_k\right) \tag{A.1}$$

$$a_i = \frac{1}{L_{ii}}\left(z_i - \sum_{k>i} L_{ki}a_k\right) \tag{A.2}$$

A.3 秘密計算の記法と要素関数

A.3.1 記法

本稿で用いる記法を表 A·2 に示す．

表 A·2 記法

記法	意味
a	平文値（公開値）
p	秘密分散値の modulus（メルセンヌ素数）
$[\![a]\!]$	$(2,3)$-Shamir 秘密分散された値（$0 \le a < p$）
$[\![\vec{a}]\!]$，$[\![A]\!]$	各要素が秘密分散されたベクトル・行列
$[\![a_i]\!]$，$[\![A_{ij}]\!]$	$[\![\vec{a}]\!]$，$[\![A]\!]$ の要素（それ自体がシェア）
$[\![A_i]\!]$	行列 $[\![A]\!]$ の第 i 行
$[\![a]\!] + [\![b]\!]$	$c := (a+b) \bmod p$ となる $[\![c]\!]$ を計算する（$-$，$-[\![a]\!]$ も同様）．ローカル計算
$c \cdot [\![a]\!]$	公開値（整数）c との積．ローカル計算
$[\![a]\!] \times [\![b]\!]$	シェア同士の乗算．関数 Mul$([\![a]\!],[\![b]\!])$ を参照（$[\![a]\!]^2 := [\![a]\!] \times [\![a]\!]$）
$[\![a]\!] > [\![b]\!]$	$c := (a > b)$ となる $[\![c]\!]$ を計算する（$\ge, <, \le$ も同様）
$[\![a]\!] \gg t$	$b := a \gg t$ となる $[\![b]\!]$ を計算する．関数 RShift$([\![a]\!], t)$ を参照
$A^\top$	転置行列
$\vec{1}$，$\vec{0}$	全要素が 1，0 である定数のベクトル
O	全要素が 0 である定数の行列（ゼロ行列）

ベクトル・行列に対する演算および関数は，要素ごとに適用する．

A.3.2 実数の扱い

基数 10，小数点位置 e の実数 r は，

$$a := r \times 10^e \tag{A.3}$$

で求められる整数 a $(\le p)$ に変換し，秘密分散する．さらに，実数系の演算では 2 を基数とした方が計算を効率化できるため，2 を基数とする表現に変換する．2 を基数とした小数点位置が $e' = 24$（24bit 固定小数点）となるようにシフト量 s のシフトを行う．

$$a' := r \times 10^e \times 2^s \tag{A.4}$$

ただし a' $(\le p)$ である．小数点位置 e，e' は平文で扱う．減算・乗算などの左右オペランドでは，基本的には e，e' の値は同じとする．

A.3.3 関数の詳細

Mul$([\![a]\!],[\![b]\!])$ は，[15] にもとづき $c := (a \times b) \bmod p$ となる $[\![c]\!]$ を返却する．ただし，a，b が指数 e' の場合は c の小数点位置が $2e'$ となっているため，小数点位置が e' となる $c' := c \gg e'$（右シフト）に対する $[\![c']\!]$ を返却する．Dot についても同様に，出力の小数点位置を入力に合わせる処理を含む．一方のオペランドが平文の固定小数点値である場合も同様に扱う．

$[\![a]\!] > [\![b]\!]$ は，$(2^\ell - 1) + [\![a]\!] - [\![b]\!]$ をビット分解し，その $\ell+1$ 番目のビットを加法的秘密分散から Shamir 秘密分散に変換することで得られる [17]．ℓ は値域から定まるビット長である．$\ge, <, \le$ は，加減およびビットの反転により同様に構成できる．

RShift$([\![a]\!], t)$ は，$[\![a]\!]$ を t ビット右シフトした値のシェアを返す．シェアは modulus p の剰余環上の値であるため，各サーバが自身のシェアをローカルに右シフトしても正しい結果は得られない．本稿では，$O(|p|)$ ビット通信量・$O(1)$ ラウンドで実行できる文献 [18] の手法を用いる．上記の Mul における小数点位置の調整（$\gg e'$）も RShift による．

なお，公開値 c による除算 $[\![a]\!]/c$ は，c が 2 べきのときは RShift$([\![a]\!], \log_2 c)$ として，一般の c のときは平文で求めた $1/c$ の固定小数点表現を Mul の一方のオペランドとすることで計算する（Algorithm 1 の step 16）．

以下に，行列・配列を並び替える関数を定義する．これらの関数は値を計算しないため，入力・出力パラメータが平文，シェアのどちらにも適用できる．

- At(X, i) は，入力の行列 X の i 列目の配列を出力する関数とする．

© 2026 Information Processing Society of Japan

- $\mathsf{Repeat}(\vec{x}, n)$ は，入力の配列 $\vec{x}$ の長さが m のとき，長さ $n \times m$ の配列 $\vec{y}$ を出力する関数とする．$y_i := x_j,\ j := \lfloor i/n \rfloor$ である．$\vec{x}$ の各要素を n 回ずつ連続して繰り返したものに相当する．
- $\mathsf{Tile}(\vec{x}, n)$ は，入力の配列 $\vec{x}$ の長さが m のとき，長さ $n \times m$ の配列 $\vec{y}$ を出力する関数とする．$y_i := x_j,\ j := i \bmod m$ である．$\vec{x}$ を n 回連結したものに相当する．

Repeat と Tile を組み合わせることで，2 つの配列の全要素に対する演算をまとめて計算できる．

A.4 ハイパーパラメータ推定手法

ハイパーパラメータ $\theta = (\ell, \sigma_f^2, \sigma_n^2)$ を求める一般的な手法を表 A·3 に示す．本稿ではハイパーパラメータ推定にグリッドサーチを採用する．

表 A·3 ハイパーパラメータ推定手法と秘密計算での課題

手法	概要	秘密計算での課題／メリット
勾配法	周辺尤度最大化．準ニュートン法(L-BFGS-B など)，複数初期値から再探索	収束判定・ライン探索で反復的な大小比較が必要．反復回数がデータに依存(サイドチャネル)
交差検証	目的関数を予測性能（LOO など）に変更	最大化自体は勾配法などに依存し同様の課題．分割数倍のコレスキー分解が必要
MCMC	事後分布のサンプリングによる完全ベイズ推定	各ステップの採択・棄却判定に秘密比較が必要．ステップ数がデータに依存
ベイズ最適化	代理モデルによる尤度近似で評価回数を抑制	代理モデルの学習・更新と獲得関数最大化という追加の反復構造が必要
座標降下法	各軸を交互に最適化（黄金分割法など）	目的関数の等高線が斜めの谷を持つと収束が遅く，反復回数がデータに依存
グリッドサーチ	固定回数の格子点評価から最良点を選択	固定回数で完結し計算の形がデータ非依存．比較・選択もシェアのまま可能（**採用**）

A.5 提案手法の下位アルゴリズム

本文 3.8 節で用いる下位アルゴリズムを示す．ハイパーパラメータ γ，η，σ_{f2} は，手動指定の場合は平文，自動推定の場合はシェアとなるが，いずれの場合も同一のアルゴリズムで扱える．

Algorithm 7 では，連立方程式 $A = K^{-1}B$，$K = LL^\top$ を解いている．$\vec{L}_{inv}$ は，Algorithm 6 に示す通り L の対角成分の逆数である．$[\![B]\!]$ は $n \times m$ 行列であり，step 3・6 の各段で長さ m の行を一括処理する．$m = 1$ のとき $[\![B]\!]$ は長さ n のベクトルとなり，Algorithm 1 の step 10 および Algorithm 2 の step 3 はこの場合にあたる．Algorithm 7 の step 3，step 6 は，それぞれ式 (A.1)，(A.2) に対応する．

Algorithm 4 BuildK_0

Functionality: $[\![K_0]\!] \leftarrow \mathsf{BuildK}_0([\![X]\!], n, d, [\![\gamma]\!])$
Input: $[\![X]\!]$，n，d，$[\![\gamma]\!]$
Output: $[\![K_0]\!]$，ただし $(K_0)_{ij} = \exp(-\gamma\|\vec{x}_i - \vec{x}_j\|^2)$

1. $[\![D]\!] := O$　// $n \times n$ のゼロ行列
2. for $k = 0, 1, \ldots, d-1$ do
3. 　$[\![\vec{c}_k]\!] := \mathsf{At}([\![X]\!], k)$
4. 　$[\![D]\!] := [\![D]\!] + \left(\mathsf{Repeat}([\![\vec{c}_k]\!], n) - \mathsf{Tile}([\![\vec{c}_k]\!], n)\right)^2$
5. $[\![K_0]\!] := \mathsf{Exp}(-[\![\gamma]\!] \times [\![D]\!])$
6. Output $[\![K_0]\!]$

Algorithm 5 $\mathsf{BuildCrossK}_0$

Functionality: $[\![K_{0,*}]\!] \leftarrow \mathsf{BuildCrossK}_0([\![X]\!], n, [\![X_*]\!], m, d, [\![\gamma]\!])$
Input: $[\![X]\!]$，n，$[\![X_*]\!]$，m，d，$[\![\gamma]\!]$
Output: $[\![K_{0,*}]\!]$，ただし $(K_{0,*})_{ij} = \exp(-\gamma\|\vec{x}_i - \vec{x}_{*,j}\|^2)$

1. $[\![D]\!] := O$　// $n \times m$ のゼロ行列
2. for $k = 0, 1, \ldots, d-1$ do
3. 　$[\![\vec{c}_k]\!] := \mathsf{At}([\![X]\!], k)$；　$[\![\vec{c}_{*,k}]\!] := \mathsf{At}([\![X_*]\!], k)$
4. 　$[\![D]\!] := [\![D]\!] + \left(\mathsf{Repeat}([\![\vec{c}_k]\!], m) - \mathsf{Tile}([\![\vec{c}_{*,k}]\!], n)\right)^2$
5. $[\![K_{0,*}]\!] := \mathsf{Exp}(-[\![\gamma]\!] \times [\![D]\!])$
6. Output $[\![K_{0,*}]\!]$

Algorithm 6 コレスキー分解

Functionality: $[\![L]\!], [\![\vec{L}_{inv}]\!] \leftarrow \mathsf{Cholesky}([\![K]\!], A_S)$
Input: $[\![K]\!]$，A_S（$\mathsf{SqrtInv}$ の値域，平文）
Output: $[\![L]\!]$，ただし $K = LL^\top$
　$[\![\vec{L}_{inv}]\!]$，ただし $L_{inv,j} := 1/L_{jj}$

1. for $j = 0, 1, \ldots, n-1$ do
2. 　$[\![v]\!] := [\![K_{jj}]\!] - \sum_{k<j} [\![L_{jk}]\!]^2$　// $j = 0$ のときは $[\![K_{jj}]\!]$
3. 　$[\![L_{inv,j}]\!] := \mathsf{SqrtInv}([\![v]\!]; A_S)$
　　$[\![L_{jj}]\!] := [\![v]\!] \times [\![L_{inv,j}]\!]$
4. 　$[\![L_{ij}]\!] := \left([\![K_{ij}]\!] - \sum_{k<j} [\![L_{ik}]\!] \times [\![L_{jk}]\!]\right) \times [\![L_{inv,j}]\!]$
　　// $i = j+1, \ldots, n-1$ を一括処理
5. Output $[\![L]\!]$，$[\![\vec{L}_{inv}]\!]$

Algorithm 7 前進後退代入

Functionality: $[\![A]\!] \leftarrow \mathsf{Solve}([\![L]\!], [\![\vec{L}_{inv}]\!], [\![B]\!])$
Input: $[\![L]\!]$，$[\![\vec{L}_{inv}]\!]$，$[\![B]\!]$
Output: $[\![A]\!]$，ただし $A = K^{-1}B$（$K = LL^\top$）

1. // 前進代入 $L[\![Z]\!] = [\![B]\!]$
2. for $i = 0, 1, \ldots, n-1$ do
3. 　$[\![Z_i]\!] := \left([\![B_i]\!] - \sum_{k<i} [\![L_{ik}]\!] \times [\![Z_k]\!]\right) \times [\![L_{inv,i}]\!]$
4. // 後退代入 $L^\top[\![A]\!] = [\![Z]\!]$
5. for $i = n-1, n-2, \ldots, 0$ do
6. 　$[\![A_i]\!] := \left([\![Z_i]\!] - \sum_{k>i} [\![L_{ki}]\!] \times [\![A_k]\!]\right) \times [\![L_{inv,i}]\!]$
7. Output $[\![A]\!]$

$1/L_{ii}$ の除算を避けるため，$[\![L_{inv,i}]\!]$ との乗算としている．また，Algorithm 6・7 の $\sum_k$ の積和は，k に関するループではなくベクトル同士の乗算 1 回と総和により求める．総和はローカル計算のため通信を伴わない．

© 2026 Information Processing Society of Japan

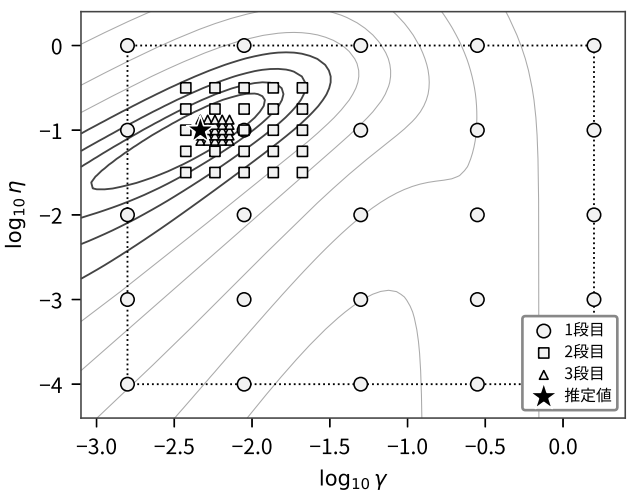


図 A·1 diabetes におけるプロファイル尤度の等高線と階層グリッド探索の格子点の例（データ分割は 4.2 節の評価とは異なる）．点線の矩形は 1 段目の探索範囲（$c_\gamma \pm 2s_\gamma$，$c_\eta \pm 2s_\eta$）を示す．

A.6 評価の詳細

A.6.1 予測値の差

表 A·4 は，テスト点ごとに平文と秘密計算の予測平均 μ_*・予測標準偏差 σ_* を比較した誤差指標である．「μ_* の標準偏差」は，テスト点間の μ_* のばらつきである．

表 A·4 予測値（μ_*・σ_*）の差

	diabetes	ccs	airfoil
RMSE(μ_*)	0.262	0.092	0.027
最大誤差 (μ_*)	1.068	0.423	0.141
RMSE(μ_*)/μ_* の標準偏差	0.47%	0.57%	0.41%
RMSE(σ_*)	0.035	0.029	0.005
最大誤差 (σ_*)	0.060	0.193	0.055

A.6.2 階層グリッド探索の挙動

図 A·1 に，diabetes のプロファイル尤度（式 (8)）の等高線と，階層グリッド探索の格子点を示す．最大値の近傍では等高線が斜めに伸びたリッジを形成しており，段を追うごとに格子がこのリッジ上に収束していることが確認できる．

A.6.3 計算量のスケーリング

ダミーデータで n（行数）・d（列数）を変化させて処理時間を測定した（train/test の区別なし）．$d = 10$ に固定して n を変化させた結果と，隣接点間の実効指数 p を表 A·5 および図 A·2 に示す．実効指数 p は，処理時間が n^p に比例すると仮定して隣接する 2 点 (n_k, t_k)，(n_{k+1}, t_{k+1}) から $p := \log(t_{k+1}/t_k)/\log(n_{k+1}/n_k)$ と定めた値であり，log-log プロット上の局所的な傾きに相当する．

表 A·5 ダミーデータによる n 依存性（$d = 10$ 固定，秒）と隣接点間の実効指数 p

n	100	500	1000	1500	2000
ハイパラ推定	90.92	514.91	1536.87	3409.91	6104.01
実効指数 p	—	1.08	1.58	1.97	2.02
fit	1.75	9.93	26.81	54.99	94.19
実効指数 p	—	1.08	1.43	1.77	1.87
predict	0.94	23.82	154.37	467.68	1141.02
実効指数 p	—	2.01	2.70	2.73	3.10

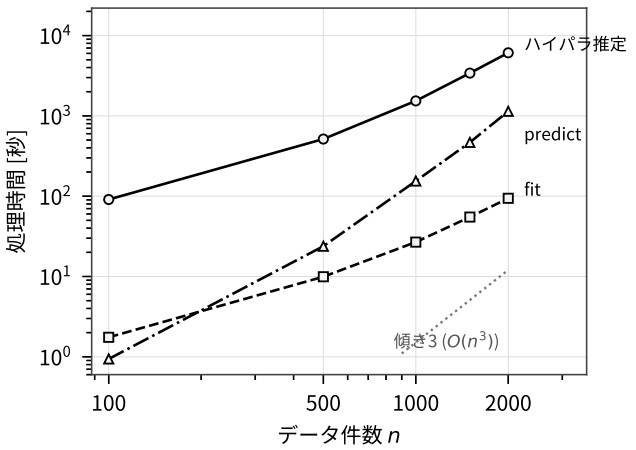


図 A·2 ダミーデータによる処理時間のスケーリング（表 A·5, $d = 10$ 固定）．点線は傾き 3（$O(n^3)$）の目安である．

n の増加とともに指数は上昇し，$n = 1500 \to 2000$ では，理論値 $O(n^3)$（predict は $O(n^2 m)$ であり，本測定では $m = n$ のため実質 $O(n^3)$）に向かって漸近する挙動であり，実データでの実効指数（1.4〜1.7 程度，通信オーバーヘッドの影響が相対的に大きい件数域）とも整合する．また，$n = 1000$ に固定して d を 5 から 50 へ 10 倍にした場合，ハイパーパラメータ推定は 1415.12 秒から 1564.87 秒と処理時間はほぼ変化しなかった（実効指数 0.03〜0.04）．これは，d が影響するのはカーネル行列を構築する際の距離計算（$O(n^2 d)$）のみであり，計算量を支配するコレスキー分解・前進後退代入が d に依存しないためである．

© 2026 Information Processing Society of Japan